Jio Platforms Limited

API SPECIFICATION DOCUMENT

Consent Management System

COPYRIGHT INFORMATION
No part of this document may be copied, reproduced, stored in any retrieval system, or transmitted in any form or by any means, either electronically, mechanically, or otherwise, without prior written permission.

© Copyright Jio Platforms Limited.
All rights reserved.
Jio Platforms Limited, Reliance Corporate Park
Thane Belapur Road, Ghansoli-400 701

TRADEMARKS
All the brand names and other products or services mentioned in this document are identified by the trademarks or service marks of their respective owners

DISCLAIMER
The information in this document is subject to change without notice and should not be construed as a commitment by Jio Platforms Limited. assumes no responsibility or makes no warranties for any errors that may appear in this document and disclaims any implied warranty of merchantability or fitness for a particular purpose

Date: November 26, 2025 *Released document | Version 2.0*

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Version History

Date	Contributor	initial Draft	Ver	State
Oct 15.2025	Racharla Vamsi	Created the document	1.1	Draft
Nov 26, 2026	Lokesh Sharma	Released Document	2.0	Released

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

References

Sr. No	Document Name and URL	Ver #
1	Business Requirement Document For Consent Management Under the DPDP Act, 2023	1.0
2	Jio Consent Management System High Level Architecture Document	1.0

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Table of Contents

Version History *1*

References *2*

Table of Contents *4*

1. Introduction *5*

2. Document Overview *6*

Assumptions 6

3. Security Layers and Data Privacy Considerations *7*

3.1 Authentication & Authorization 7

3.2 Transport Security 7

3.3 Data Protection & Storage 7

3.4 Event Integrity & Recovery 7

3.5 Privacy-by-Design Features 7

3.6 Data Retention and Minimization 7

4. Detailed API Specification *8*

4.1 Authentication and Security 8

4.1.1 Token 8

4.1.2 Acquisition (OAuth2 Client Credentials Flow) 8

4.2 Secure Code APIs 9

4.2.1 Secure Code Create 9

4.3 Consent APIs 10

4.3.1 Consent Handle Create 10

4.3.2 Consent Token Validation 12

4.4 Grievance APIs 13

4.4.1 Get Grievance by ID 13

4.4.2 Update Grievance Status 14

4.5 Notification APIs – Missed Notifications 15

4.5.1 Get All Missed Notifications (All) 15

4.5.2 Get Missed Notification by ID 17

4.5.3 Update Missed Notification Status 18

4.5.4 List Missed Notifications 19

Appendix: Error Codes and Response Structure *21*

Appendix: What is x-jws-signature *22*

Appendix: Glossary *23*

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

1. Introduction

Jio Consent Management System (CMS) is a plug-and-play module that integrates with Data Fiduciary (DF) systems to enable DPDP-compliant consent and grievance lifecycle management, rather than serving as a centralized consent manager for multiple data fiduciaries. Consent Management System (CMS) facilitates secure, standardized, and auditable interactions among three key stakeholders: Data Principals (users), Data Fiduciaries (DFs), and Processors.

In addition to its modular deployment capabilities, CMS emphasizes strong security and privacy principles to ensure that Consent Management System (CMS) helps data fiduciaries systems to meet regulatory compliance while safeguarding the privacy and trust of Data Principals. This document is meant to provide detailed API specifications required for this integration.

CMS APIs are essential for:

- Allowing DFs to comply with DPDP obligations seamlessly
- Keeping DFs and DPs updated and synchronized on consent/grievance lifecycle events
- Automating digital consent and grievance workflows within DF platforms
- Ensuring traceability, reliability, and recoverability of critical user decisions

"CONFIDENTIAL – FOR NeGD(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

2. Document Overview

This document provides comprehensive API specifications for integrating data fiduciary (DF) and third-party data processor (DP) systems with the Jio consent management system (CMS). This document includes:

1) Security Layers and Data Privacy Considerations
2) Authentication and token handling
3) Callback registration and event ping notifications
4) Event payload retrieval using a pull model
5) Token validation APIs
6) Missed event recovery
7) Error handling and response codes
8) Glossary and terminology
9) Digital Signature process to sign the consent artifact and all API responses.

Assumptions

1) Each data fiduciary (DF)/ data processor (DP) system is pre-registered and provisioned with OAuth2 credentials.
2) Data fiduciary (DF)/ Data processor (DP) systems are capable of securely receiving callbacks and making REST API calls.
3) Consent Management System (CMS) and consumer systems synchronize on event_id for end-to-end tracking.

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

3. Security Layers and Data Privacy Considerations

The CMS uses multiple layers of security and follows built-in data privacy rules to meet the requirements of the DPDP Act. Below are the core security and privacy controls:

3.1 Authentication & Authorization

- All API endpoints are protected using OAuth 2.0 (client credentials grant)
- Role-based access controls (RBAC) ensure that DFs and DPs access only permitted resources
- Tokens are short-lived and scoped to prevent misuse

3.2 Transport Security

- All communication must occur over HTTPS with TLS 1.3
- Callback notifications are signed using JWS to ensure integrity and authenticity

3.3 Data Protection & Storage

- Personally identifiable information (PII) is stored in encrypted form using AES-256.
- Consent artifacts are signed using JWS keys and can be validated by consumers
- Access logs and audit trails are immutable and encrypted

Commented [MOU1]:

Commented [MOU2]:

Commented [MOU3]:

3.4 Event Integrity & Recovery

- All callback payloads are idempotent and traceable using event IDs
- Missed delivery tracking and retry queues are implemented
- Dead Letter Queue (DLQ) support enables the reliable recovery of failed notifications

3.5 Privacy-by-Design Features

- No sensitive user data is embedded in callback payloads
- Multilingual support and WCAG accessibility compliance are built in for user communications
- DLT-compliant templates for SMS notifications and masking in email templates

3.6 Data Retention and Minimization

- Consent and grievance records are retained only as per defined regulatory timelines.
- Data is regularly purged/anonymized beyond retention thresholds

"CONFIDENTIAL - FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

4. Detailed API Specification

All API's in this ecosystem are secured with detached JWS signature, where the entire HTTP body request is signed using the CCA private key and passed as value for x-jws-signature in the header.

4.1 Authentication and Security

This section describes how third-party systems authenticate securely with CMS before calling any API. All APIs require a valid OAuth2 Bearer token obtained via the token API. All API interactions with CMS require OAuth2 Bearer Tokens. CMS uses a centralized Identity and Authentication Service to issue and validate tokens to DF and DPs.

4.1.1 Token

4.1.2 Acquisition (OAuth2 Client Credentials Flow)

This API is intended to get an access token for API authorization.

Attribute	Details
Method	POST
Endpoint	/oauth2/token
Purpose	Obtain an access token for API authorization (client-to-server)

a) Request Headers

Header	Value
Content-Type	application/x-www-form-urlencoded

(Optionally, OAuth2 deployments may also use Authorization: Basic <base64(client_id:client_secret)>.)

b) Request Body

Parameter	Type	Required	Description
grant_type	string	Y	Must be client_credentials

In some deployments, client_id and client_secret may be sent in headers or configuration instead of body.

c) Request Body Example

```
{
  "grant_type": "client_credentials"
}
```

d) Response Body Parameters

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Field	Type	Description
access_token	string	Token to be used for API authorization
token_type	string	Token type, always Bearer
expires_in	integer	Validity period in seconds
scope	string	Space-separated list of scopes, for example default

e) Response Body Example

```
{
  "access_token": "eyJ...",
  "scope": "default",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

f) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity (where enabled)

4.2 Secure Code APIs

4.2.1 Secure Code Create

This API creates a secureCode for a given identity and identityType.

Attribute	Details
Method	POST
Endpoint	/secureCode/create
Purpose	Create a secureCode for a given identity (e.g., mobile number)

a) Request Headers

Header	Value
Content-Type	application/json
Authorization	Bearer <access_token>
tenant-Id	Tenant identifier
business-Id	Business identifier within the tenant
requestor-type	It will be DATA_FIDUCIARY

(Bearer token obtained via /oauth2/token.)

b) Request Body

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Parameter	Type	Required	Description
identityValue	string	Y	Identity value (e.g., mobile number)
identityType	string	Y	Type of identity (e.g., MOBILE)

c) Request Body Example

```
{
  "identityValue": "8668260292",
  "identityType": "MOBILE"
}
```

d) Response Body Parameters

Field	Type	Description
secureCode	string	Generated secure code identifier
identity	string	Identity for which the secureCode was created
expiry	integer	Expiry timestamp (epoch milliseconds)

e) Response Body Example

```
{
  "secureCode": "9ba0fe25-3f0b-4600-8580-8716977daf12",
  "identity": "8668260292",
  "expiry": 1762937192373
}
```

f) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

4.3 Consent APIs

4.3.1 Consent Handle Create

This API creates a consent handle based on a consent template and customer identifiers.

Attribute	Details
Method	POST
Endpoint	/consent/v1.0/consent-handle/create
Purpose	Create a consent handle for a Data Principal using a configured template

a) Request Headers

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Header	Required	Description
txn	Y	Transaction ID (UUID) for traceability
Tenant-Id	Y	Tenant identifier
Business-Id	Y	Business identifier within the tenant
Authorization	Y	Bearer <access_token>

b) Request Body

Parameter	Type	Required	Description
templateId	string	Y	Identifier of the consent template
templateVersion	number	Y	Version of the consent template
customerIdentifiers	object	Y	Customer identifier object

customerIdentifiers object

Field	Type	Required	Description
type	string	Y	Type of identifier (e.g., MOBILE)
value	string	Y	Value of identifier (e.g., mobile number)

c) Request Body Example

```
{
  "templateId": "TEMPL000004883951",
  "templateVersion": 1,
  "customerIdentifiers": {
    "type": "MOBILE",
    "value": "9999999999"
  }
}
```

d) Response Body Parameters

The specification only defines that a consent handle is created with HTTP 200.
The detailed response structure is implementation-specific.

Field	Type	Description
(consentHandle)	object	Consent handle payload as per implementation

e) Response Body Example

```
{
  "consentHandleId": "HANDLE-123456",
  "status": "CREATED"
}
```

(Example structure; adjust to actual implementation if you have a concrete response.)

f) Response Headers

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

4.3.2 Consent Token Validation

This API validates a consent token.

Attribute	Details
Method	POST
Endpoint	/consent/v1.0/validate-token
Purpose	Validate a consent token received as part of consent workflows

a) Request Headers

Header	Required	Description
consent-token	Y	Consent token to be validated (string)
txn	Y	Transaction ID (UUID) for traceability
tenant-id	Y	Tenant identifier
Authorization	Y	Bearer <access_token>

b) Request Body

Parameter	Type	Required	Description
consentToken	string	Y	Consent token value to be validated

c) Request Body Example

```
{
 "consentToken": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9..."
}
```

d) Response Body Parameters

The specification indicates a 200 response on success; the body is not explicitly defined.

Field	Type	Description
valid	boolean	Indicates if the token is valid
reason	string	Reason or message (if any)

(Structure above is indicative; align with your actual implementation.)

e) Response Body Example

```
{
 "valid": true,
```

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

"reason": "Token is valid and active"
}

f) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

4.4 Grievance APIs

4.4.1 Get Grievance by ID

This API retrieves grievance details by grievance ID.

Attribute	Details
Method	GET
Endpoint	/grievance/api/v1/grievances/{id}
Purpose	Retrieve grievance details by ID

a) Request Headers

Header	Required	Description
X-Tenant-Id	Y	Tenant identifier
X-Business-Id	Y	Business identifier
X-Transaction-Id	Y	Transaction identifier
Authorization	Y	Bearer <access_token>

b) Path Parameters

Parameter	Type	Required	Description
id	string	Y	Grievance ID

c) Request Body

Not applicable (GET).

d) Response Body Parameters

Response schema is not detailed in the YAML; it is implementation-specific.

Field	Type	Description
id	string	Grievance ID
status	string	Current status of the grievance

"CONFIDENTIAL - FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Field	Type	Description
createdAt	string	Creation timestamp
updatedAt	string	Last updated timestamp
details	object	Additional grievance details

(Adjust fields to your actual implementation.)

e) Response Body Example


```
{
  "id": "GRV-12345",
  "status": "OPEN",
  "createdAt": "2025-11-01T10:00:00Z",
  "updatedAt": "2025-11-02T12:30:00Z",
  "details": {
    "category": "CONSENT_ISSUE",
    "description": "User reported incorrect consent status."
  }
}
```


f) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

4.4.2 Update Grievance Status

This API updates the status and resolution remark of a grievance.

Attribute	Details
Method	PUT
Endpoint	/grievance/api/v1/grievances/{id}
Purpose	Update grievance status and resolution remark

a) Request Headers

Header	Required	Description
X-Tenant-Id	Y	Tenant identifier
X-Business-Id	Y	Business identifier
X-Transaction-Id	Y	Transaction identifier
Authorization	Y	Bearer <access_token>

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

b) Path Parameters

Parameter	Type	Required	Description
id	string	Y	Grievance ID

c) Request Body

Parameter	Type	Required	Description
status	string	Y	New grievance status (e.g., RESOLVED)
resolutionRemark	string	Y	Remarks regarding resolution

d) Request Body Example

```
{
 "status": "RESOLVED",
 "resolutionRemark": "Forwarded to municipal officer"
}
```

e) Response Body Parameters

The YAML describes a 200 "Status updated" with no explicit schema.
Typically, a small acknowledgement payload is returned.

Field	Type	Description
id	string	Grievance ID
status	string	Updated status
resolutionRemark	string	Resolution remark

f) Response Body Example

```
{
 "id": "GRV-12345",
 "status": "RESOLVED",
 "resolutionRemark": "Forwarded to municipal officer"
}
```

g) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

4.5 Notification APIs – Missed Notifications

4.5.1 Get All Missed Notifications (All)

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Attribute	Details
Method	GET
Endpoint	/notification/v1/missed-notifications/all
Purpose	Retrieve all missed events for a recipient

a) Request Headers

Header	Required	Description
X-Tenant-Id	Y	Tenant identifier
X-Business-Id	Y	Business identifier
Authorization	Y	Bearer <access_token>
X-Recipient-Type	Y	Recipient type (e.g., DATA_FIDUCIARY)
X-Recipient-Id	Y	Recipient ID

b) Path / Query Parameters

None.

c) Request Body

Not applicable (GET).

d) Response Body Parameters

YAML defines only description, not schema; consider this a list of missed events.

Field	Type	Description
items	array	List of missed notification objects

e) Response Body Example

```
[
 {
  "id": "MISS-001",
  "eventType": "CONSENT_UPDATE",
  "createdAt": "2025-11-01T12:00:00Z",
  "status": "PENDING"
 }
]
```

f) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

4.5.2 Get Missed Notification by ID

Attribute	Details
Method	GET
Endpoint	/notification/v1/missed-notifications/{id}
Purpose	Retrieve a specific missed notification event by ID

a) Request Headers

Header	Required	Description
X-Tenant-Id	Y	Tenant identifier
X-Business-Id	Y	Business identifier
Authorization	Y	Bearer <access_token>
X-Recipient-Type	Y	Recipient type
X-Recipient-Id	Y	Recipient identifier

b) Path Parameters

Parameter	Type	Required	Description
id	string	Y	Missed notification ID

c) Request Body

Not applicable (GET).

d) Response Body Parameters

Implementation-specific missed notification object.

Field	Type	Description
id	string	Missed notification ID
eventType	string	Event type
createdAt	string	Creation timestamp
status	string	Status of missed notification

e) Response Body Example

```
{
  "id": "MISS-001",
  "eventType": "CONSENT_UPDATE",
  "createdAt": "2025-11-01T12:00:00Z",
  "status": "PENDING"
}
```

g) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability

"CONFIDENTIAL - FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Header	Description
x-jws-signature	JWS for response authenticity

4.5.3 Update Missed Notification Status

Attribute	Details
Method	PUT
Endpoint	/notification/v1/missed-notifications/{id}/status
Purpose	Update the status of a missed notification

a) Request Headers

Header	Required	Description
X-Tenant-Id	Y	Tenant identifier
X-Business-Id	Y	Business identifier
Authorization	Y	Bearer <access_token>
X-Recipient-Type	Y	Recipient type
X-Recipient-Id	Y	Recipient ID

b) Path Parameters

Parameter	Type	Required	Description
id	string	Y	Missed notification ID

c) Request Body

Parameter	Type	Required	Description
status	string	Y	Updated status (e.g., ACKNOWLEDGED)
acknowledgementId	string	N	Acknowledgement reference
remark	string	N	Additional remark

d) Request Body Example

```
{
 "status": "ACKNOWLEDGED",
 "acknowledgementId": "string",
 "remark": "string"
}
```

e) Response Body Parameters

200 response with "Status updated"; schema is not detailed.
A simple echo or acknowledgement is expected.

Field	Type	Description
id	string	Missed notification ID

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Field	Type	Description
status	string	Updated status
acknowledgementId	string	Acknowledgement reference
remark	string	Optional remark

f) Response Body Example


```
{
 "id": "MISS-001",
 "status": "ACKNOWLEDGED",
 "acknowledgementId": "ACK-12345",
 "remark": "Processed successfully"
}
```


g) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

4.5.4 List Missed Notifications

Attribute	Details
Method	GET
Endpoint	/notification/v1/missed-notifications/list
Purpose	Retrieve a list of missed notifications for a recipient

a) Request Headers

Header	Required	Description
X-Tenant-Id	Y	Tenant identifier
X-Business-Id	Y	Business identifier
Authorization	Y	Bearer <access_token>
X-Recipient-Type	Y	Recipient type
X-Recipient-Id	Y	Recipient ID

b) Path / Query Parameters

None.

c) Request Body

Not applicable (GET).

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

d) Response Body Parameters

Similar to /all; returns a list of missed notification objects.

Field	Type	Description
items	array	List of missed notifications

e) Response Body Example


```
[
 {
  "id": "MISS-001",
  "eventType": "CONSENT_UPDATE",
  "createdAt": "2025-11-01T12:00:00Z",
  "status": "PENDING"
 },
 {
  "id": "MISS-002",
  "eventType": "GRIEVANCE_STATUS",
  "createdAt": "2025-11-02T09:15:00Z",
  "status": "PENDING"
 }
]
```


f) Response Headers

Header	Description
Content-Type	application/json
x-correlation-id	UUID for traceability
x-jws-signature	JWS for response authenticity

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix: Error Codes and Response Structure

Code	Meaning	Description
200	OK	Request was successful
201	Created	The request was successful and a new resource was created.
202	Accepted	The request has been accepted for processing, but the processing is not yet complete.
400	Bad Request	Input is malformed or missing fields
401	Unauthorized	Access token is missing or invalid
403	Forbidden	Caller not permitted to access the resource
404	Not Found	Event ID or resource was not found
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server failed to process the request

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix: What is x-jws-signature

It is a detached JSON Web Signature (JWS) sent in the HTTP header of an API request or response. This signature is generated using the sender's private key and follows the RS256 algorithm (RSA Signature with SHA-256). The actual data (payload) is sent in the HTTP body, while the signature is sent separately in the x-jws-signature header.

How It Works

Sender Side:

- Serialize the payload (usually JSON) to plain text.
- Generate a detached JWS signature using the private key.
- Set the signature in the x-jws-signature HTTP header.
- Send the payload in the HTTP body.

Receiver Side:

- Extract the signature from the x-jws-signature header.
- Extract the payload from the HTTP body.
- Verify the signature using the sender's public key.
- If valid, proceed with processing the request.

Why this matters in CMS

- Ensures data integrity: The payload cannot be tampered with.
- Provides non-repudiation: The sender cannot deny having sent the data.
- Keeps payload and signature separate for efficiency, infra load and clarity.

"CONFIDENTIAL – FOR NeGD USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix: Glossary

Term	Description
DF	Data Fiduciary (organization collecting user consent)
Processors	Processor (third party acting on behalf of DF)
CMS	Consent Management System
JWT	JSON Web Token, signed artifact for tamper-proof transmission
Callback	Webhook triggered by CMS on DF/DP endpoint
Event ID	Unique identifier representing consent or grievance event
DLQ	Dead Letter Queue (for failed notification delivery recovery)
Artifact	A digitally signed consent record, issued by CMS

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"